

Tecnológico de Monterrey

Evidencia 2

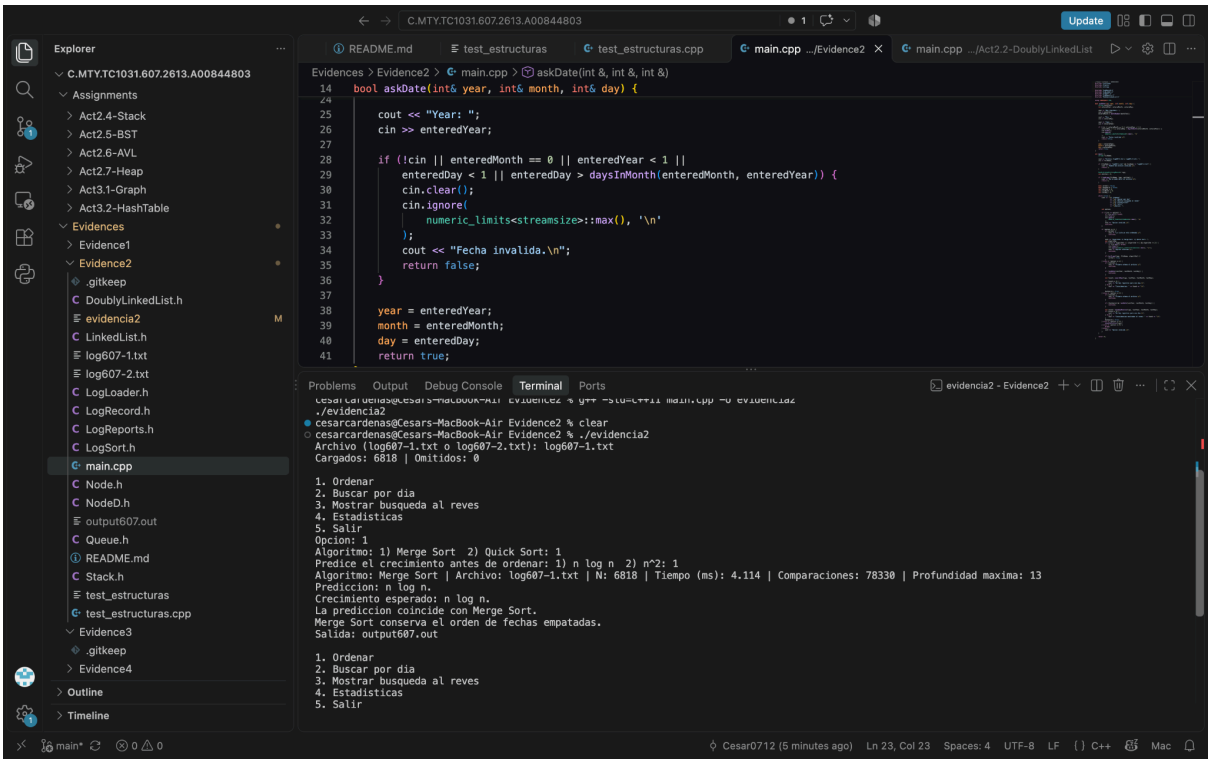
Programación de estructuras de datos y algoritmos fundamentales (Gpo 607)

Estudiante: César Aldahir Cárdenas Navarro

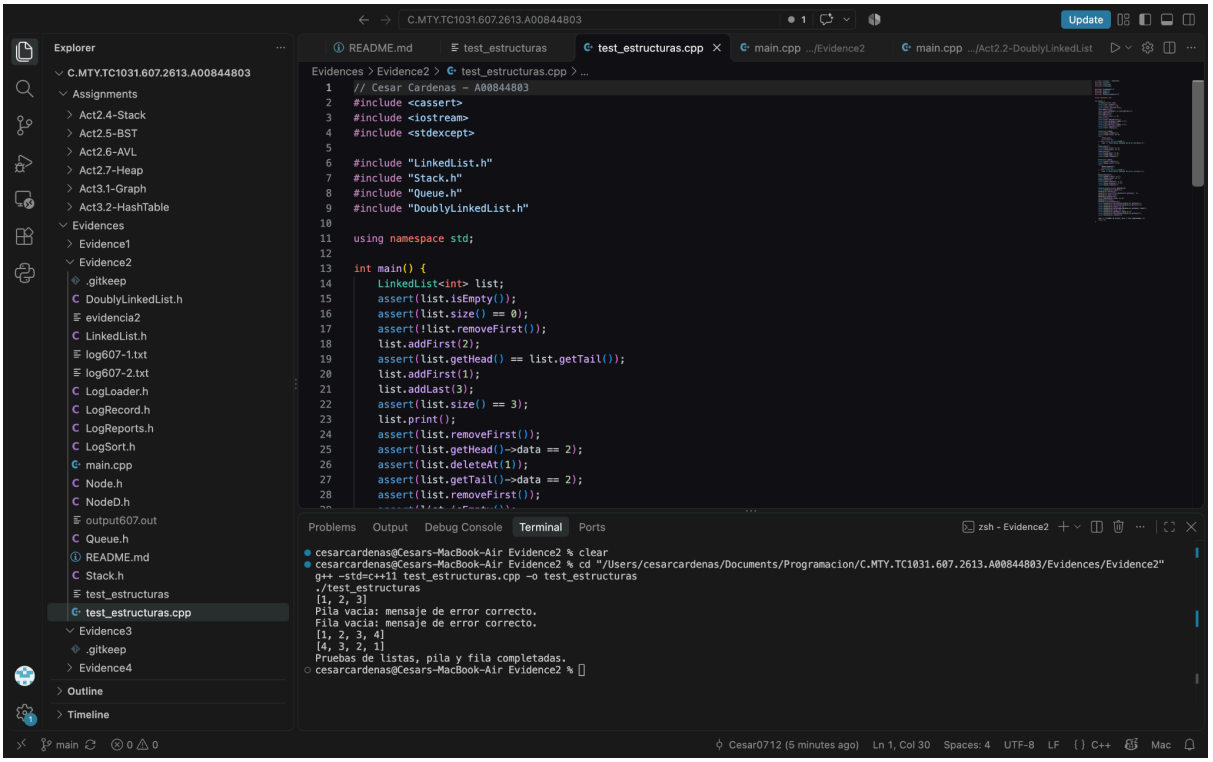
Matrícula: A00844803

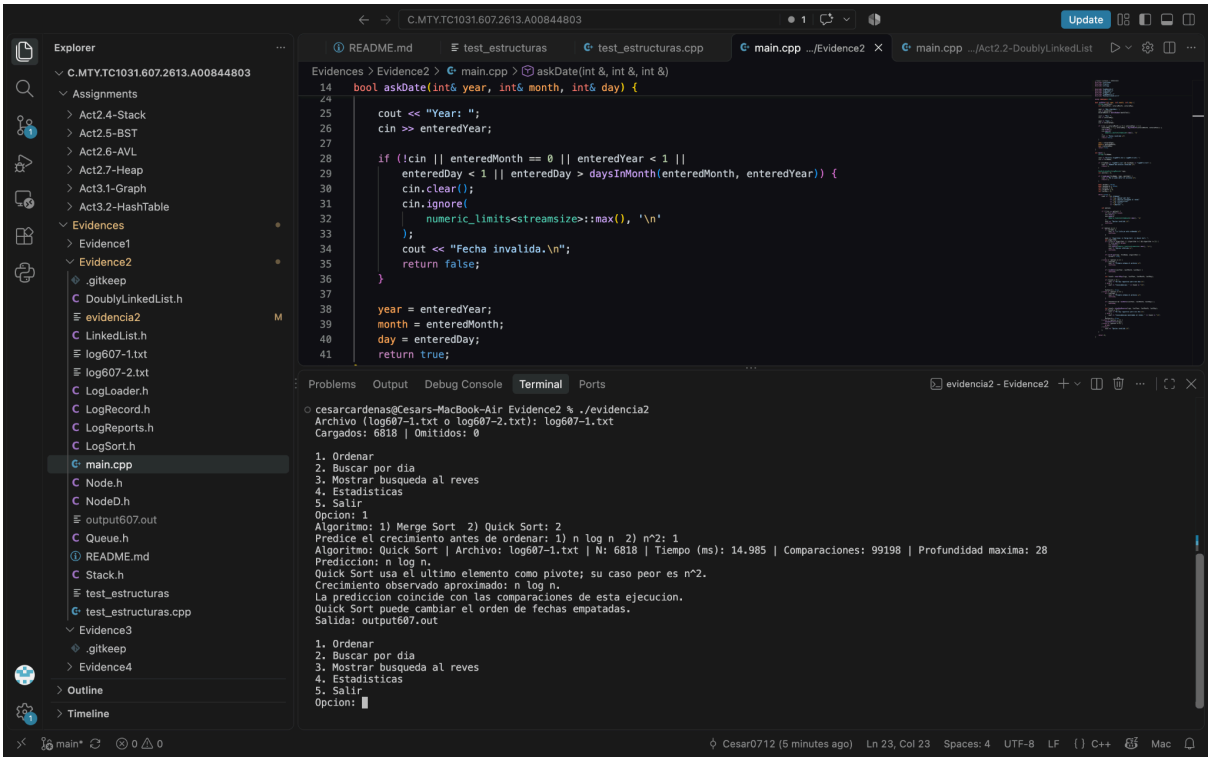
Fecha: 18/10/2026

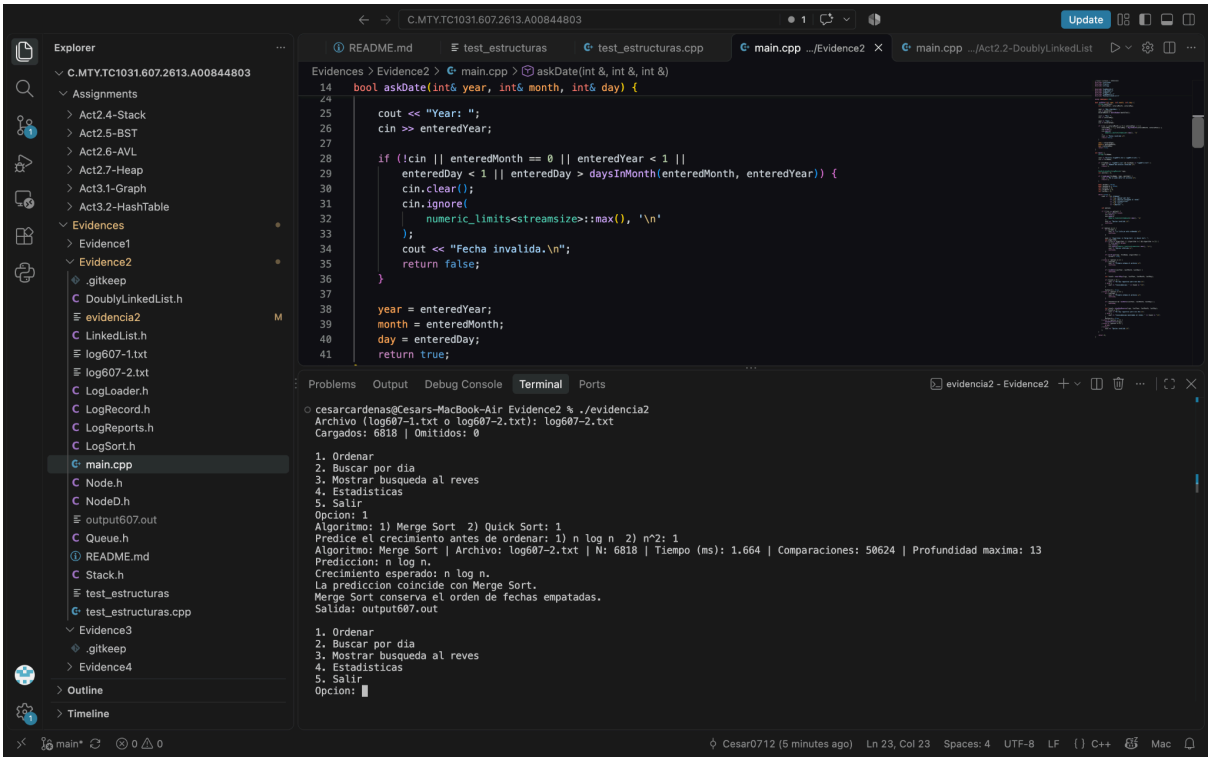
Prueba de estructuras

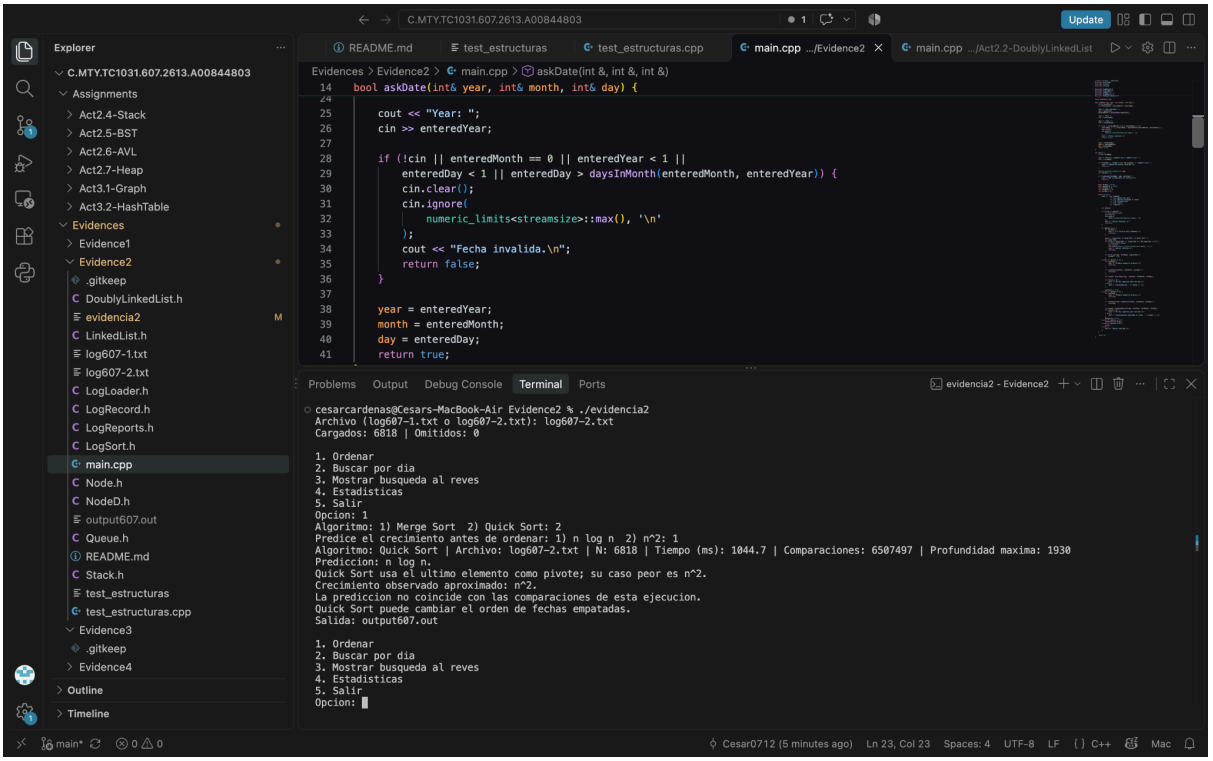


Ordenamiento

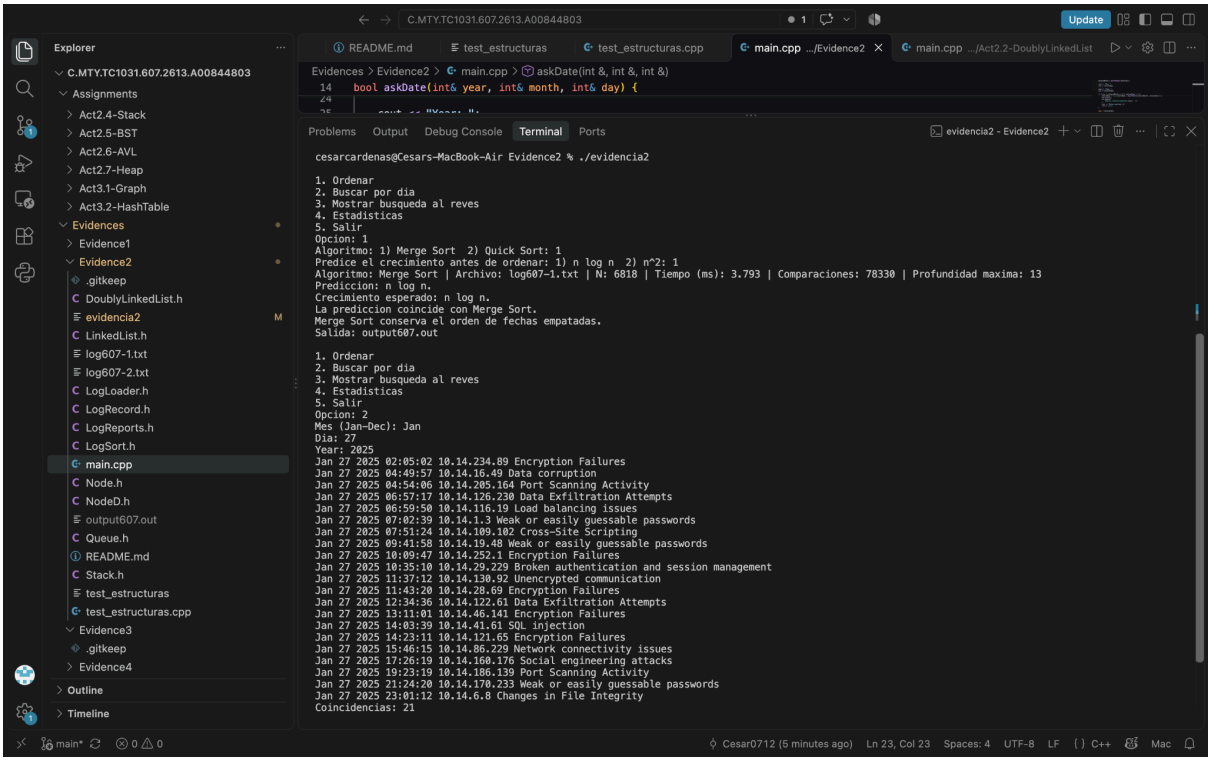



C.MTY.TC1031.607.2613.A00844803
Update
Explorer
C.MTY.TC1031.607.2613.A00844803
Assignments
Act2.4-Stack
Act2.5-BST
Act2.6-AVL
Act2.7-Heap
Act3.1-Graph
Act3.2-HashTable
Evidences
Evidence1
Evidence2
.gitkeep
DoublyLinkedList.h
evidencia2
LinkedList.h
log607-1.txt
log607-2.txt
LogLoader.h
LogRecord.h
LogReports.h
LogSort.h
main.cpp
Node.h
NodeD.h
output607.out
Queue.h
README.md
Stack.h
test_estructuras
test_estructuras.cpp
Evidence3
.gitkeep
Evidence4
Outline
Timeline
README.md
test_estructuras
test_estructuras.cpp
main.cpp .../Evidence2
main.cpp .../Act2.2-DoublyLinkedList
Evidences > Evidence2 > main.cpp > askDate(int &, int &, int &)
bool askDate(int& year, int& month, int& day) {
cout << "Year: ";
cin >> enteredYear;
if (!cin || enteredMonth == 0 || enteredYear < 1 ||
enteredDay < 1 || enteredDay > daysInMonth(enteredMonth, enteredYear)) {
cin.clear();
cin.ignore(
numeric_limits<streamsize>::max(), '\n'
);
cout << "Fecha invalida.\n";
return false;
}
year = enteredYear;
month = enteredMonth;
day = enteredDay;
return true;
Problems
Output
Debug Console
Terminal
Ports
evidencia2 - Evidence2
cesarcardenas@Cesars-MacBook-Air Evidence2 % ./evidencia2
Archivo (log607-1.txt o log607-2.txt): log607-1.txt
Cargados: 6818 | Omitidos: 0
1. Ordenar
2. Buscar por dia
3. Mostrar busqueda al reves
4. Estadisticas
5. Salir
Opcion: 1
Algoritmo: 1) Merge Sort 2) Quick Sort: 2
Predice el crecimiento antes de ordenar: 1) n log n 2) n^2: 1
Algoritmo: Quick Sort | Archivo: log607-1.txt | N: 6818 | Tiempo (ms): 14.985 | Comparaciones: 99198 | Profundidad maxima: 28
Prediccion: n log n.
Quick Sort usa el ultimo elemento como pivote; su caso peor es n^2.
Crecimiento observado aproximado: n log n.
La prediccion coincide con las comparaciones de esta ejecucion.
Quick Sort puede cambiar el orden de fechas empatadas.
Salida: output607.out
1. Ordenar
2. Buscar por dia
3. Mostrar busqueda al reves
4. Estadisticas
5. Salir
Opcion:
main*
0 0
Cesar0712 (5 minutes ago)
Ln 23, Col 23
Spaces: 4
UTF-8
LF
C++
Mac

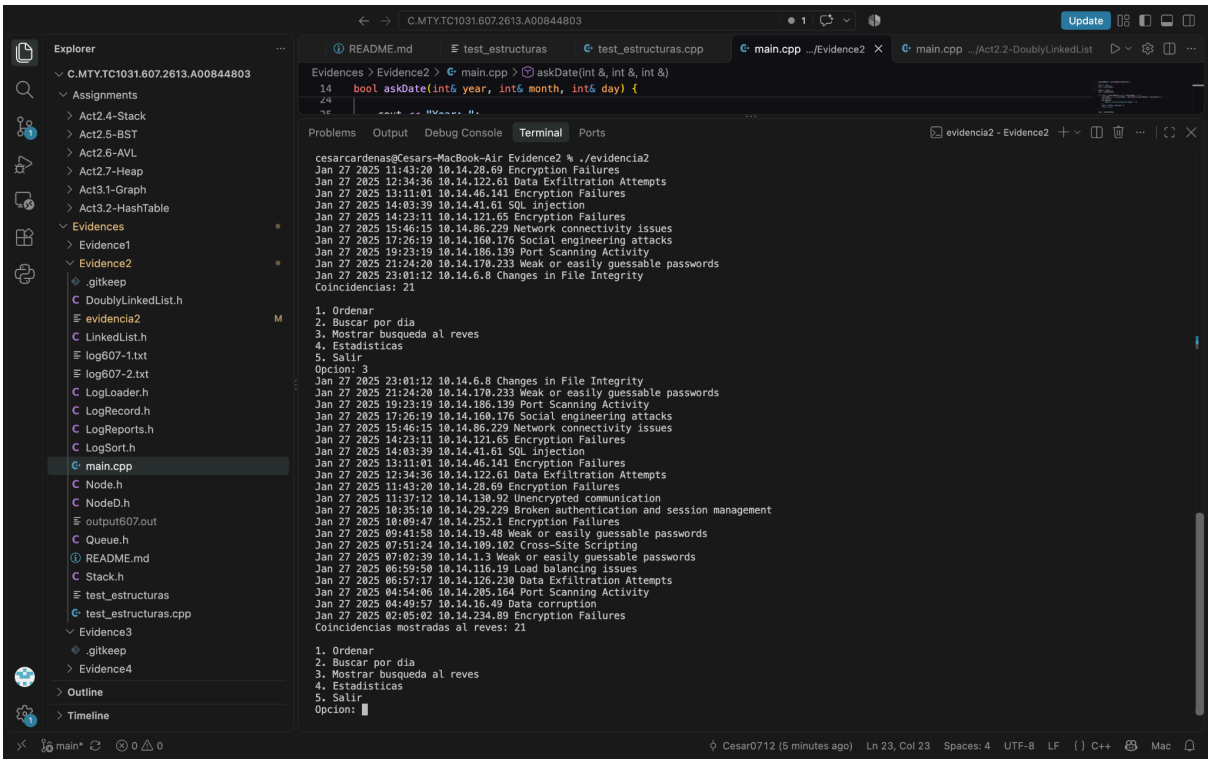

C.MTY.TC1031.607.2613.A00844803
Update
Explorer
C.MTY.TC1031.607.2613.A00844803
Assignments
Act2.4-Stack
Act2.5-BST
Act2.6-AVL
Act2.7-Heap
Act3.1-Graph
Act3.2-HashTable
Evidences
Evidence1
Evidence2
.gitkeep
DoublyLinkedList.h
evidencia2
LinkedList.h
log607-1.txt
log607-2.txt
LogLoader.h
LogRecord.h
LogReports.h
LogSort.h
main.cpp
Node.h
NodeD.h
output607.out
Queue.h
README.md
Stack.h
test_estructuras
test_estructuras.cpp
Evidence3
.gitkeep
Evidence4
Outline
Timeline
README.md
test_estructuras
test_estructuras.cpp
main.cpp .../Evidence2
main.cpp .../Act2.2-DoublyLinkedList
Evidences > Evidence2 > main.cpp > askDate(int &, int &, int &)
bool askDate(int& year, int& month, int& day) {
cout << "Year: ";
cin >> enteredYear;
if (!cin || enteredMonth == 0 || enteredYear < 1 ||
enteredDay < 1 || enteredDay > daysInMonth(enteredMonth, enteredYear)) {
cin.clear();
cin.ignore(
numeric_limits<streamsize>::max(), '\n'
);
cout << "Fecha invalida.\n";
return false;
}
year = enteredYear;
month = enteredMonth;
day = enteredDay;
return true;
Problems
Output
Debug Console
Terminal
Ports
evidencia2 - Evidence2
cesarcardenas@Cesars-MacBook-Air Evidence2 % ./evidencia2
Archivo (log607-1.txt o log607-2.txt): log607-2.txt
Cargados: 6818 | Omitidos: 0
1. Ordenar
2. Buscar por dia
3. Mostrar busqueda al reves
4. Estadisticas
5. Salir
Opcion: 1
Algoritmo: 1) Merge Sort 2) Quick Sort: 1
Predice el crecimiento antes de ordenar: 1) n log n 2) n^2: 1
Algoritmo: Merge Sort | Archivo: log607-2.txt | N: 6818 | Tiempo (ms): 1.664 | Comparaciones: 50624 | Profundidad maxima: 13
Prediccion: n log n.
Crecimiento esperado: n log n.
La prediccion coincide con Merge Sort.
Merge Sort conserva el orden de fechas empatadas.
Salida: output607.out
1. Ordenar
2. Buscar por dia
3. Mostrar busqueda al reves
4. Estadisticas
5. Salir
Opcion:
main*
0 0
Cesar0712 (5 minutes ago)
Ln 23, Col 23
Spaces: 4
UTF-8
LF
C++
Mac



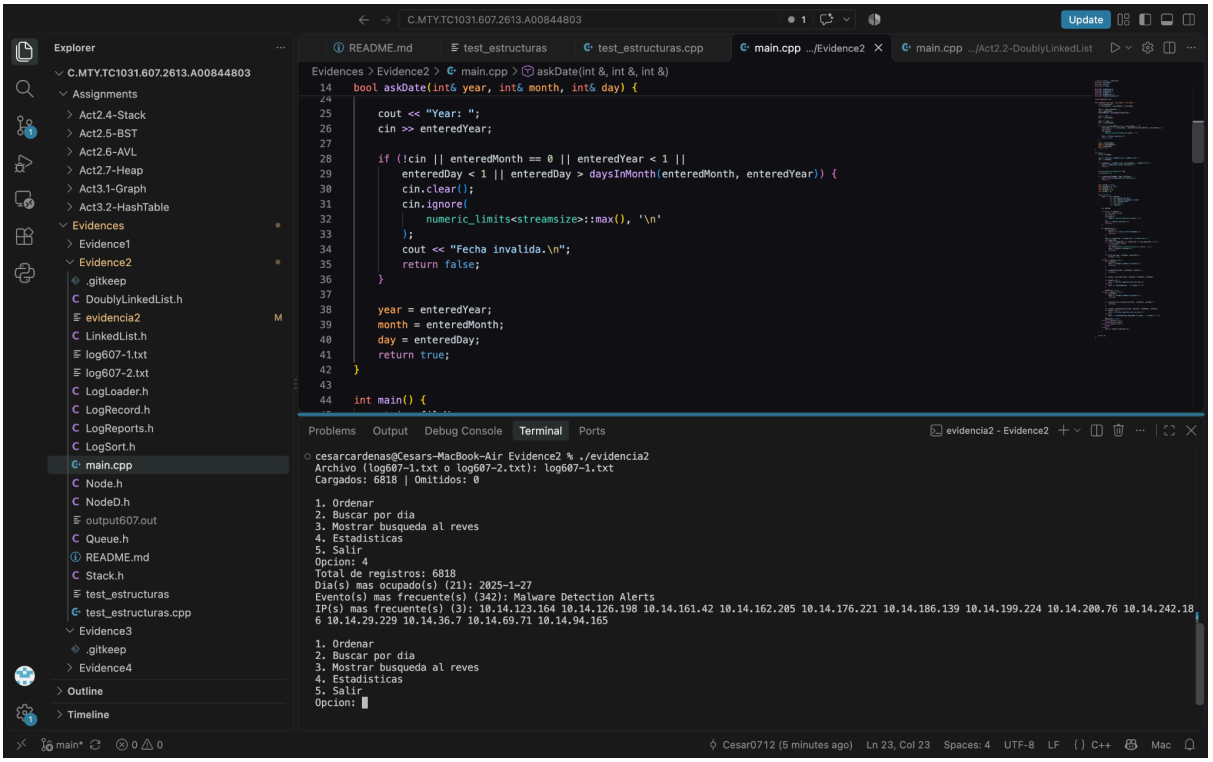
Busqueda por dia



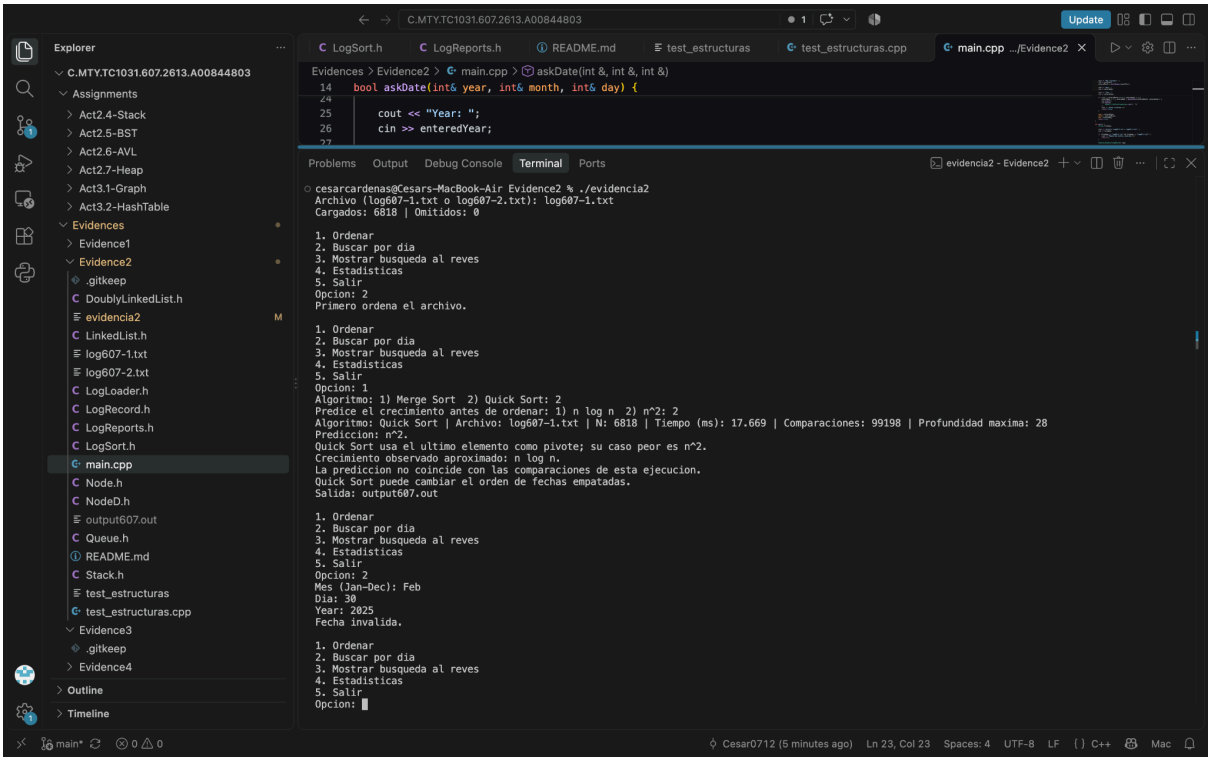
Búsqueda inversa con pila



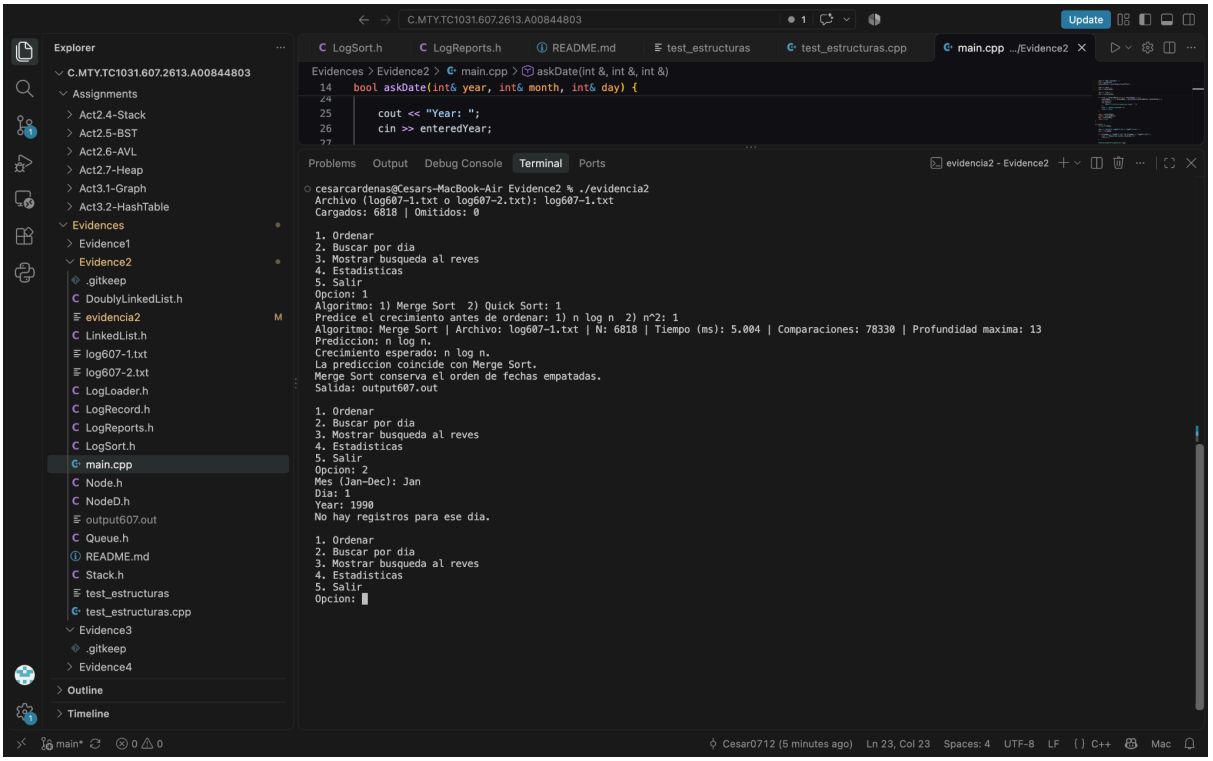
Estadísticas



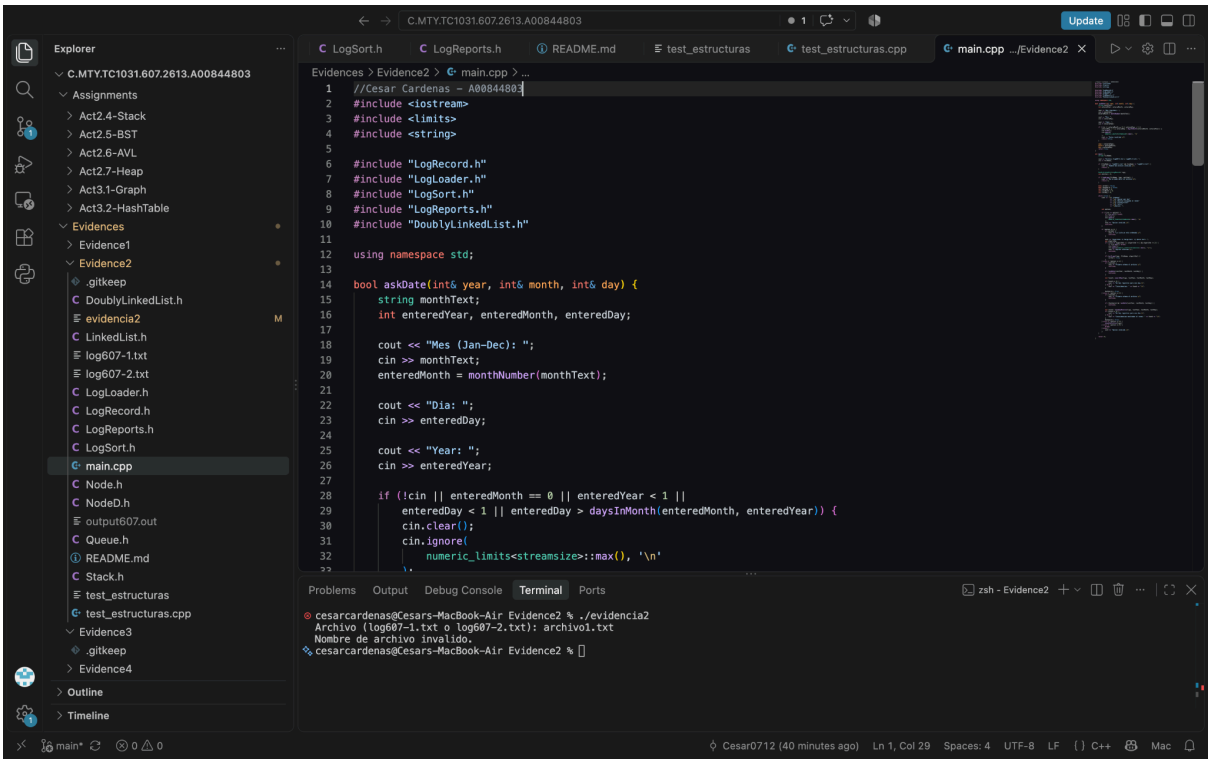
Validación de opciones y fechas



Búsqueda sin resultados



Archivo no encontrado



Interpretación de Resultados

Archivo	Algoritmo	Predicción	¿Coincidió?	Tiempo de esta prueba (ms)	Comparaciones	Profundidad máxima	Estable
log607-1.txt	Merge Sort	n log n	Sí	4.114	78330	13	Sí (conserva orden)
log607-1.txt	Quick Sort	n log n	Sí	14.985	99198	28	No (puede cambiar orden)
log607-2.txt	Merge Sort	n log n	Sí	1.664	50624	13	Sí (conserva orden)
log607-2.txt	Quick Sort	n log n	No	1044.7	6507497	1930	No (puede cambiar orden)

Están presentes las cuatro combinaciones solicitadas (Merge Sort y Quick Sort aplicados a los archivos log607-1.txt y log607-2.txt).

Merge Sort: Presenta un comportamiento totalmente consistente con el crecimiento esperado O(n log n) en ambos archivos. Muestra tiempos de ejecución bajos (4.114 ms y 1.664 ms), un número reducido de comparaciones y una profundidad máxima de recurrencia estable de 13. La predicción coincidió en ambas pruebas. Además, es un algoritmo estable que conserva el orden original de registros con fechas empatadas.

Quick Sort: En el archivo log607-1.txt su predicción (n log n) coincidió con el rendimiento observado (14.985 ms y 99,198 comparaciones). Sin embargo, en el archivo log607-2.txt la predicción no coincidió, ya que al utilizar el último elemento como pivote el algoritmo cayó en su peor caso cuadrático O(n^2), disparando el tiempo a 1044.7 ms, las comparaciones a 6,507,497 y la profundidad máxima a 1930. Asimismo, Quick Sort no garantiza estabilidad y puede alterar el orden relativo de elementos empatados.

Prompts utilizados

- "¿Cuál es la diferencia entre Merge Sort y Quick Sort al ordenar mis registros?"
- "¿Qué errores o casos límite debo revisar al cargar los archivos `log607`?"
- "Revisa mi programa según los requisitos de la actividad y explícame qué errores o casos incompletos encuentras."

Reflexión personal

Utilicé ChatGPT como apoyo para aclarar dudas, explicarme conceptos y revisar mi programa contra los requisitos de la actividad. Yo implementé la base de las estructuras (Queue, Stack y DoublyLinkedList), la carga y validación de los logs, los ordenamientos, la búsqueda por día, las estadísticas y el menú, y yo ejecuté las pruebas y tomé las mediciones. La IA me ayudó a entender cómo aprovechar el orden de la lista para buscar un día, cómo funciona la pila para invertir resultados y qué significan las comparaciones y la profundidad máxima. Aprendí que sirve para entender ideas y detectar detalles, pero sus explicaciones y sugerencias hay que comprobarlas ejecutando el código. Sin su ayuda habría tardado más revisando programas pasados y probando casos, y quizá habría revisado menos casos límite, como fechas inválidas o archivos faltantes.

Lugar de cada estructura

La Queue se usa al cargar el archivo porque las líneas deben procesarse en el mismo orden en que aparecen (FIFO), lo que me permite reportar el número de línea de las omitidas. La DoublyLinkedList guarda los registros validados porque crece sin tamaño fijo, permite agregar al final en O(1) y se puede ordenar con Merge Sort enlazando nodos sin copiar datos. La Pila se usa en la opción 3 para invertir los resultados del día.

¿Era necesaria la Pila? No estrictamente. Como la lista tiene prev, podría ubicar el último registro del día y recorrerlo hacia atrás, sin copias y con memoria adicional O(1). La pila copia cada LogRecord (con sus cadenas), usa memoria O(k) para k coincidencias y agrega

el trabajo de apilar y desapilar. Su ventaja es que separa los resultados del resto de la lista, no depende de que la estructura tenga enlaces hacia atrás y muestra claramente el comportamiento LIFO. La opción con prev es más eficiente, pero depende de la lista doble y exige localizar bien el último nodo del día. Con 6,818 registros la diferencia práctica es mínima, así que elegí la pila por claridad.

Búsqueda binaria

Necesita acceso directo al elemento del medio. En una lista ligada llegar al nodo central cuesta recorrer n/2 nodos, y en cada paso siguiente hay que recorrer de nuevo; al sumar n/2 + n/4 + ... el costo total es O(n), igual que una búsqueda lineal. Hacerla posible exigiría construir un arreglo auxiliar de apuntadores a los nodos (O(n) de tiempo y memoria, y reconstruirlo cada vez que la lista cambie) o usar otra estructura, como un árbol balanceado o una skip list. Por eso mi búsqueda recorre desde el inicio y se detiene en cuanto pasa del día buscado.

Mediciones

Como muestra la tabla, Merge Sort mantuvo una profundidad de 13, consistente con log2(6,818) ≈ 12.7, y usó menos comparaciones en log607-2 porque el archivo está casi ordenado y al mezclar una de las mitades se agota pronto. Quick Sort se comportó bien en log607-1, pero en log607-2 mi predicción de n log n no coincidió: al usar el último elemento como pivote, en datos casi ordenados el pivote cae cerca de un extremo, las particiones quedan desbalanceadas y el comportamiento se acerca a $O(n^2)$. La profundidad de recursión es el número de llamadas apiladas en la pila de llamadas: 13 marcos en Merge Sort contra 1,930 en Quick Sort, y en el peor caso serían n, con riesgo de desbordar la pila con más datos. Un pivote aleatorio o la mediana de tres lo mitigaría, y Merge Sort además es estable con timestamps repetidos.